

Introduction to Database Systems

Homework 3: Design Improvements and Tuning

Eleni Tziritza Zacharatou and Omar Shahbaz Khan

Fall 2024

1 SQL DDL [30 points]

Write SQL DDL commands to create the hiking database of Homework 2. Your script must implement the official solution ER diagram posted on LearnIT, run in PostgreSQL as a whole, and use the methodology in the textbook and lectures. The relations must include all relevant primary key, candidate key, foreign key, and NOT NULL constraints. If these standard constraints cannot enforce certain restrictions, you can omit them (but if you are having fun, feel free to implement queries to detect violations or triggers to prevent them). Please keep in mind that your SQL DDL script should address any additional requirements from Homework 2 that couldn't be represented in the ER diagram but can be implemented in the DDL (e.g., additional keys). Select reasonable types for attributes.

2 Practical Normalization [30 points]

In the homework quiz on LearnIT, you will find a script to create and populate five independent relations, each having seven columns and a primary key with a few thousand rows. For this project, *use only the Rentals relation!*

Each relation models a potential real-life database design situation, with some design flaws that must be addressed. In short, each of the relations has embedded a set of functional dependencies, which are not stated explicitly. You must a) find these dependencies and b) use them to guide the decomposition of the relations to 3NF/BCNF normal forms, using the methodology covered in class.

Note: In the homework quiz on LearnIT, you can find some additional material (slides, code, and video) to help you get started with this exercise.

2.1 Steps

More specifically, for the Rental relation, take the following steps:

- (a) Find all the important FDs in the relations, given the constraints and simplifying assumptions that are listed in detail below.

Note: We strongly recommend using your favorite programming language to generate an SQL script with all the FD detection queries, using the SQL query template covered in slide 44 of the provided slides for this quiz. The program could take as input a list of column names, and output all the required SQL queries into a text file. You can then run the text file using `psql`. Figure 1 shows example Python code to get you started.

- (b) Decompose the relation until each sub-relation is in 3NF/BCNF, while preserving all non-redundant FDs. Write down the resulting schema description in a simple Relation(columns) format.
- (c) Write the detailed SQL commands to create the resulting tables (with primary and foreign keys) and populate them, by extracting the relevant data from the original relations.

Note: In this homework, use the “homework” version of commands to create the new relations (see slide 9 of the provided slides for this quiz), so that you can compare the result of the decomposition with the original relation. Specifically, you should not run any ALTER TABLE commands.

- (d) Select the correct normal form for the decomposed schema.

Note: You can find a video from a past edition of the course explaining the process on LearnIT.

```
-----
def PrintSQL(Att1, Att2)
    print ("Here make the SQL query to check %s --> %s" % (Att1, Att2));
    print ("Use the query in slide 44 of the homework slides on LearnIT");

R = ['A', 'B', 'C']
for i in range(len(R)):
    for j in range(len(R)):
        if (i != j):
            PrintSQL(R[i], R[j])
-----
```

Figur 1: Python skeleton for code to check all FDs for a relation $R(A, B, C)$ with three attributes in step (a).

2.2 Simplifying Assumptions = Reality

In this project, assume that the following simplifying assumptions hold for each of the relations (this is the “reality” against which you check them):

- The relations must each be considered in isolation. The columns have short names that hint at their meaning, but you should not count on implicit FDs derived from the expected meaning of the columns. In short, the column names may be misleading.
- Assume that all functional dependencies in each relation (aside from primary key dependencies and dependencies derived from that) can be found by checking only FDs with one column on each side, such as $A \rightarrow B$.

Note: You can detect potential FDs using SQL queries. Using the assumptions above, it is indeed relatively simple to create a program to output queries for all possible checks for FDs, which you can then run with `psql`, thus automating the detection process.

- If you find a functional dependency that holds for the given instance, you can assume that it will hold for all instances of the relation.

Exception: When an ID column and a corresponding string column both determine each other, consider that only the ID column determines the string column, not vice versa. For example, if both $CID \rightarrow CN$ and $CN \rightarrow CID$ are found to potentially hold, then consider that only $CID \rightarrow CN$ is valid.

- The only dependencies you need to consider for decomposition are a) the dependencies that can be extracted from the data based on the assumptions above, and b) the given key constraints. As covered in the lecture, you can ignore trivial, unavoidable, and redundant functional dependencies.

3 Index Selection [20 points]

Consider the following relation with information on university courses:

Course (course_id, course_name, teacher_id, semester, ECTS, ...)

You are given a set of *unclustered* B+-tree indexes for the Course relation. For each query, identify the index (or indexes) that a good query optimizer is most likely to use. If a full table scan would yield better performance than any available index, select “no index”.

3.1 Available Indexes

The available indexes are:

- (a) Course(course_id)

- (b) Course(course_name)
- (c) Course(teacher_id)
- (d) Course(semester)
- (e) Course(ECTS)
- (f) Course(teacher_id, semester)
- (g) Course(semester, ECTS)
- (h) No index

3.2 Queries

The queries are:

Query 1:

```
select course_id
  from Course
 where teacher_id = 101;
```

Query 2:

```
select course_id, ECTS
  from Course
 where semester = 'Fall 2024';
```

Query 3:

```
select course_id, course_name
  from Course
 where ECTS > 7;
```

Query 4:

```
select course_id
  from Course
 where teacher_id = 101 and semester = (select max(semester) from Course);
```

Query 5:

```
select course_id, course_name
  from Course;
```

4 ER Diagram [5 points]

See multiple choice questions on the LearnIT quiz.

5 Passing Grade

The maximum grade is 85 points. You can see the detailed breakdown of points on the LearnIT quiz. To pass the homework, you must have *at least 25/85* points.